

April 28, 2026

```
[105]: # Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_CEILING, ROUND_HALF_UP, getcontext #
↳ Besonders für sig. Runden
import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
import scipy.constants as c
from scipy.stats import norm
from scipy.optimize import curve_fit
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.integrate import quad
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelmin, argrelmax
from scipy.special import factorial

%matplotlib widget
import matplotlib.pyplot as plt
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path
from pathlib import Path

import pandas as pd # Auch wichtig für den Latex Export
from pytexit import py2tex
```

```

import csv
import re
from typing import List

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Math, Latex, HTML

# Grafiken in EU-Größen ausgeben
def figsize_cm(width_cm, height_cm):
    return (2*width_cm / 2.54, 2*height_cm / 2.54)
def comma_to_float(valstr):
    return float(valstr.replace(',', '.'))

g=9.80984
Delta_g=0.00002

```

```

[106]: currentversuch = "223"
base = Path.cwd()
Messwerte_path = Path(base.parent.parent.parent/"Messwerte"/currentversuch)
    ↪ # initializing paths
Ausgabe_path = Path(base/"Diagramme")
print(Messwerte_path)
print(Ausgabe_path)

```

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2.1\Messwerte\223

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2.1\Auswertungen\Code\223\Diagramme

Runden signifikanter Stellen

```

[107]: def round_to_sigs(val, errVal, einheiten):
    """
        Funktion zur Rundung eines Fehlers und die Anpassung des Messwertes daran.
        ↪ Diese Funktion wurde etwas umständlicher
        geschrieben, da python mit Floats und Runden schnell in Probleme rennt.
        ↪ Daher musste hier mit dezimal gearbeitet werden.
        Zudem sollten besonders kleine und große Messwerte in der
        ↪ Dezimalschreibweise geschrieben werden, damit diese auch
        für Protokolle geeignet sind.

        Parameter
        -----
        **val** : float
            Messwert
    """

```

```

**errVal** : float
    Ungenauigkeit des Messwertes

Return
-----

**value_rounded** : str
    Gerundeter Messwert

**error_rounded** : str
    Gerundete Ungenauigkeit des Messwertes

**res** : str
    "Messwert \pm Fehler"
    """

# Daten zu Dezimal wechseln, da Floats probleme machen
val = Decimal(str(val))
errVal = Decimal(str(errVal))

exp = int(math.floor(math.log10(float(errVal)))) # Die erste
↳ signifikante Stellenposition wird anhand des errVal bestimmt

    if round(float(errVal / (Decimal(10) ** exp))) < 3: # wenn 1,2,3
↳ erste signifikante Stelle, dann kommt eine zweite Nachkommastellenposition
↳ hinzu
        exp -= 1

scale = Decimal(10) ** exp

val_round = (val / scale).quantize(Decimal('1'), rounding=ROUND_HALF_UP) *
↳ scale # mit scale wird das Komma so verschoben, dass alle
↳ signifikanten Stellen vor dem Komma stehen, und dann alle Nachkommastellen
↳ weggerundet werden
err_round = (errVal / scale).quantize(Decimal('1'), rounding=ROUND_CEILING)
↳ * scale

qty = f"\qty{{{val_round}}({err_round})}{{{einheiten}}}"
num = f"\num{{{val_round}}({err_round})}"
return float(val_round), float(err_round), num, qty

v_round = np.vectorize(round_to_sigs, otypes=[float, float, object, object],
↳ excluded=['einheiten'])

# wissenschaftliche Notation erstmal vernachlässigen (da häufig Einheiten
↳ gewollt sind, die 10^e Prefixe beinhalten)

```

Gausssche Fehlerfortpflanzung

```
[108]: def gff(function, errPronePar, latex=True):
    """
    Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

    Parameters
    -----
    **func** : sympy function
        Funktion dessen Fehler bestimmt werden soll.

    **errPronePar** : Array
        Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
        Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind
    ↪ ungleich den Werten für x, y, z.
        Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc.
    ↪ genutzt und für deren Fehler err_x, err_y, err_z etc.

    Return
    -----
    **absolut_err** : sympy function
        Gibt die Fehlergleichung des absoluten Fehlers wieder.

    **relativ_err** : sympy function
        Gibt die Fehlergleichung des relativen Fehlers wieder.

    **errProneParameters** : array
        Liste aller Fehlerbehafteten Größen
    """

    error = 0
    errProneParameters = []

    function = sp.sympify(function)
    variables = errPronePar.split(",")
    variables = sp.symbols(variables)

    for variable in variables:
        Delta = sp.Symbol(f"Delta_{variable}")
        partial = sp.diff(function, variable) # Die Funktion
    ↪ wird nach der fehlerbehafteten Variable abgeleitet
        term = sp.UnevaluatedExpr(partial*Delta)**2
        error = error + ((term)) # Fehler werden
    ↪ quadratisch aufsummiert
        errProneParameters.append((variable, Delta))

    absolute_err = sp.simplify(sp.sqrt(error), rational = True)
```

```

relative_err=sp.simplify(sp.sqrt(error/function**2),rational = True)

if latex:
    #Prints out the Latex Code for the Functions
    print("Funktion:")
    display(Math(sp.latex(function,long_frac_ratio=2)))
    print(sp.latex(function))
    print("Absoluter Fehler:")
    display(Math(sp.latex(absolute_err,long_frac_ratio=2)))
    print(sp.latex(absolute_err))
    print("Relativer Fehler:")
    display(Math(sp.latex(relative_err,long_frac_ratio=2)))
    print(sp.latex(relative_err))
    return function,absolute_err, relative_err, errProneParamaters

```

Sigma-Abweichungen

```

[109]: def sigma_abweichung(p1, err_p1,p2,err_p2):
        """
        Funktion zum berechnen der Sigma-Abweichung von zwei Messwerten, oder einem
        ↪Messwert und einem Literaturwert.
        """
        if err_p1 == 0 and err_p2 == 0:
            raise ValueError("Für die Sigma-Abweichung muss mindestens ein Wert
            ↪fehlerbehaftet sein!")
        else:
            abweichung=Decimal(str(abs(p1 - p2)/(np.sqrt(err_p1 ** 2+err_p2 ** 2))))

        if abweichung == 0:
            return 0.0, "\\num{0}"

        exp = int(math.floor(math.log10(float(abweichung))))
        if round(float(abweichung / (Decimal(10) ** exp))) < 3:           # wenn
        ↪1,2,3 erste signifikante Stelle, dann kommt eine zweite
        ↪Nachkommastellenposition hinzu
            exp -= 1
        scale = Decimal(10) ** exp
        abweichung_round = (abweichung / scale).quantize(Decimal('1'),
        ↪rounding=ROUND_CEILING) * scale
        res = f"\\num{{{abweichung_round}}}"

        return float(abweichung_round),res

```

```

[110]: # #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2

```

```

#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
import textwrap
def compare_table(nam1,nam2,einheiten,val1,err1,val2,err2):
    VAL1=round_to_sigs(val1,err1,r'[2]
    VAL2=round_to_sigs(val2,err2,r'[2]
    abs=np.abs(val1-val2)
    abs_delta=np.sqrt(err1**2+err2**2)
    if abs_delta!=0:
        SIGMA=sigma_abweichung(val1,err1,val2,err2)[1]
    else:
        SIGMA=0.0
    ABS=round_to_sigs(abs,abs_delta,r'[2]
    rel=abs/np.abs(val1)*100
    rel_delta=rel*np.sqrt((abs_delta/abs)**2+(err1/val1)**2) if abs != 0 else 0
    REL=round_to_sigs(rel,rel_delta,r'[2]
    output = textwrap.dedent(f"""
        \\begin{{table}}[htb]
        \\centering
        \\caption{{}}
        \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}
        \\toprule
        \\
        ↪{{nam1}}[\\unit{{{{einheiten}}}}]&{{nam2}}[\\unit{{{{einheiten}}}}]&\\text{{abs.Abw.
        ↪}}[\\unit{{{{einheiten}}}}]&\\text{{proz.Abw.
        ↪}}[\\unit{{{{percent}}}}]&S[\\sigma]\\\\\\midrule
        {VAL1}&{VAL2}&{ABS}&{REL}&{SIGMA}\\\\\\
        \\bottomrule
        \\end{{tabularx}}
        \\label{{table:Vergleich_{{nam1}}}}
        \\end{{table}}
    """)
    print(output)

```

```

[111]: # #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

```

```

#Erstellung von Vergleichstabellen in Latex
def compare_table_withliterature(nam1,nam2,einheiten,val1,err1,val2):
    VAL1=round_to_sigs(val1,err1,r'[2]
    abs=np.abs(val1-val2)
    abs_delta=np.sqrt(err1**2)
    if abs_delta!=0:
        SIGMA=sigma_abweichung(val1,err1,val2,0)[1]
    else:
        SIGMA=0.0
    ABS=round_to_sigs(abs,abs_delta,r'[2]
    rel=abs/np.abs(val2)*100
    rel_delta=rel*np.sqrt((abs_delta/abs)**2+(err1/val1)**2) if abs != 0 else 0
    REL=round_to_sigs(rel,rel_delta,r'[2]
    output = textwrap.dedent(f"""
        \\begin{{table}}[htb]
        \\centering
        \\caption{{}}
        \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}
        \\toprule
        \\
        \\rightarrow{{nam1}}[\\unit{{{{einheiten}}}}]&{{nam2}}[\\unit{{{{einheiten}}}}]&\\text{{abs.Abw.
        \\rightarrow}}[\\unit{{{{einheiten}}}}]&\\text{{proz.Abw.
        \\rightarrow}}[\\unit{{{{\\percent}}}}]&S[\\sigma]\\\\\\midrule
        \\{VAL1}&{val2}&{ABS}&{REL}&{SIGMA}\\\\\\
        \\bottomrule
        \\end{{tabularx}}
        \\label{{table:Vergleich_{{nam1}}}}
        \\end{{table}}
    """)
    print(output)

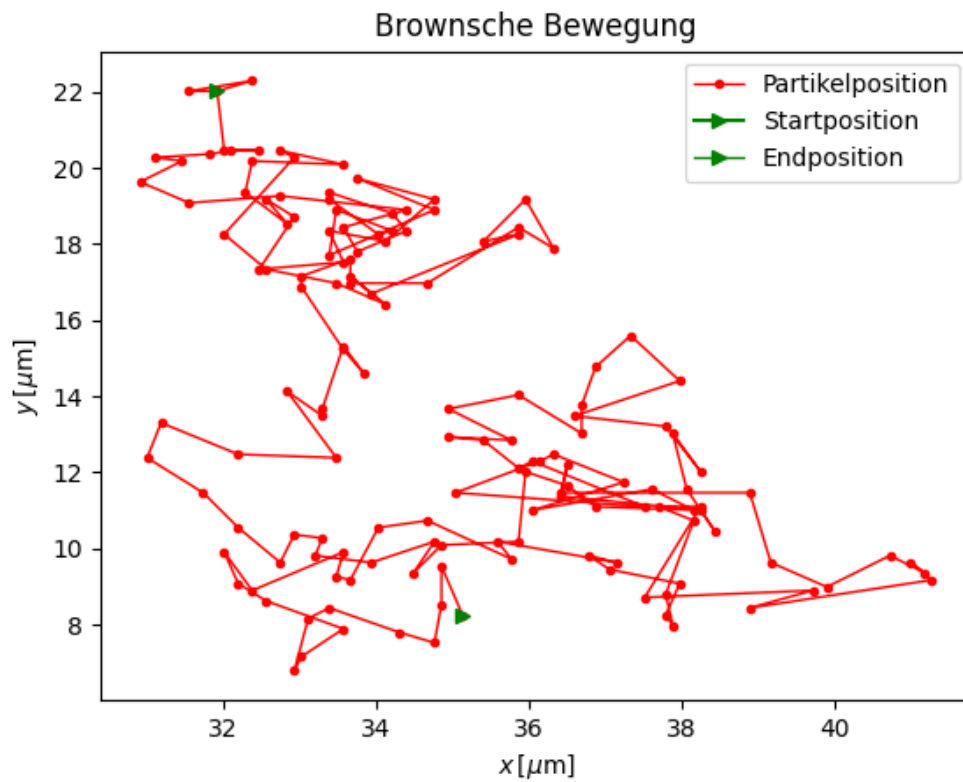
```

```

[173]: plt.close('all')
t,x,y = np.loadtxt(Messwerte_path/"position_data.
    ↳txt",delimiter=",",dtype="float", unpack=True)
plt.plot(x,y, marker='.', color='red', linewidth=1, label='Partikelposition')
plt.plot(x[0],y[0], marker='>', color='green', label='Startposition')
plt.plot(x[149],y[149], marker='>', color='green',
    ↳linewidth=1,label='Endposition')
plt.xlabel(r'$x\,[\mu m]$')
plt.ylabel(r'$y\,[\mu m]$')
plt.title('Brownsche Bewegung')
plt.legend()

plt.show()
plt.savefig(Ausgabe_path/"Nr.1.Graph.png",format="png")

```



```
[181]: import re

data1="""
# M 416 921.5
# L 416 860
# L 361.5 860
# L 307 860
# L 307 849.49
# L 307 838.99
# L 371.25 839.24
# L 435.5 839.5
# L 435.76 911.25
# L 436.01 983
# L 426.01 983
# L 416 983
# M 416 921.5

"""
data2="""
M 485 974.98
```



```

C 476.49 971.01 471.39 964.09 470.32 955.06
C 469.11 944.83 469.08 944.88 517.45 889.81
C 541.88 862 563.87 837.61 566.31 835.61
C 568.75 833.6 584.42 824.51 601.12 815.39
C 617.83 806.28 632.29 798.12 633.25 797.26
C 635.34 795.39 635.52 792.67 633.65 791.12
C 632.87 790.48 608.39 787.86 575.75 784.93
L 519.19 779.86
L 461.42 804.6
C 408.88 827.1 403.15 829.33 398.08 829.32
C 376.59 829.28 366.09 802.58 381.73 787.74
C 383.64 785.92 403.85 776.69 440.57 760.87
C 471.33 747.61 498.98 735.89 502 734.82
C 512.93 730.95 514.47 730.97 578.5 735.92
C 612.05 738.51 639.53 740.6 639.57 740.57
C 640.05 740.14 693.03 612.71 692.81 612.52
C 692.64 612.38 678.55 602.04 661.5 589.55
C 644.45 577.05 629.22 565.63 627.65 564.16
C 626.09 562.7 623.34 558.58 621.54 555
C 613.39 538.77 576.16 457.19 575.71 454.57
C 574.39 446.92 580.25 436.58 587.72 433.38
C 595.65 429.98 607.05 433.09 611.52 439.87
C 612.84 441.87 623.4 463.95 635 488.95
C 646.6 513.95 656.62 535.04 657.26 535.82
C 658.71 537.56 709.84 574.02 719.5 580.19
C 723.35 582.65 738.65 591.03 753.5 598.82
L 780.5 612.97
L 821.21 646.26
C 843.6 664.58 863.54 681.64 865.52 684.19
C 868.89 688.51 870.11 692.46 883.6 742.66
C 899.09 800.27 899.05 800.06 894.95 808.09
C 888.77 820.21 868.91 821.37 861.42 810.05
C 859.7 807.46 855.02 791.54 845.81 756.92
L 832.65 707.5
L 806.75 686.15
C 792.51 674.41 780.71 664.96 780.52 665.15
C 780.34 665.34 767.57 695.59 752.14 732.37
C 736.72 769.15 722.9 800.98 721.44 803.09
C 719.97 805.21 717.16 808.16 715.2 809.66
C 713.23 811.16 686.62 825.54 656.06 841.61
C 625.5 857.68 598.83 872.24 596.78 873.95
C 594.73 875.66 574.35 898.36 551.48 924.38
C 523.23 956.51 508.6 972.42 505.84 973.97
C 499.96 977.27 490.86 977.72 485 974.98
M 485 974.98
""""
data3=""

```

```

M 764.27 588.1
C 751.75 583.46 742.28 573.67 737.96 560.92
C 736.22 555.76 735.87 552.83 736.19 545.98
C 736.93 530.25 745.43 517.36 759.72 510.29
C 765.99 507.2 768.39 506.57 775.4 506.2
C 789.36 505.47 799.42 509.47 808.93 519.52
C 828.35 540.06 822.03 573.5 796.34 586.15
C 790.29 589.13 788.61 589.49 779.56 589.75
C 771.56 589.98 768.43 589.64 764.27 588.1
M 764.27 588.1
"""
data = [data1, data2, data3]
pattern = r'([MLC])\s*([\d\s.-]+)'      # ([]) ist Gruppe, speichert in Gruppe1
    ↳ den Buchstaben [M oder L oder C] ab      ↳ * beliebig viele Leerzeichen
    ↳ () Gruppe zwei

plt.close('all')

for i in range(3):
    x=[]
    y=[]
    for command, values in re.findall(pattern, data[i]):
        # Alle Zahlen im Block extrahieren und in Floats umwandeln
        nums = [float(n) for n in re.findall(r'[-+]?[d*\.]?[d+]', values)] #[-+]??:
        ↳ negative Zahlen      ↳ d*: beliebig viele Ziffern      ↳ .?: optionaler
        ↳ Dezimalpunkt      ↳ d+: Sucht nach mindestens einer Ziffer nach dem Punkt und
        ↳ wird von leerzeichen gestoppt

        if command == 'M' or command == 'L':
            # Nimm das erste (und einzige) Paar: [x, y]
            x.append(nums[0])
            y.append(nums[1])

        elif command == 'C':
            # Ein C-Befehl hat 6 Zahlen (x1, y1, x2, y2, x3, y3)
            # Wir wollen nur das letzte Paar (Index 4 und 5)
            x.append(nums[4])
            y.append(nums[5])
    x=np.array(x)
    y=1123-np.array(y)
    if i == 0:
        # Nur beim ersten Durchgang das Label setzen
        plt.plot(x, y, marker='.', color='red', linewidth=1, label='particle
        ↳ position')
    else:
        # Danach ohne Label plotten
        plt.plot(x, y, marker='.', color='red', linewidth=1)

```

```

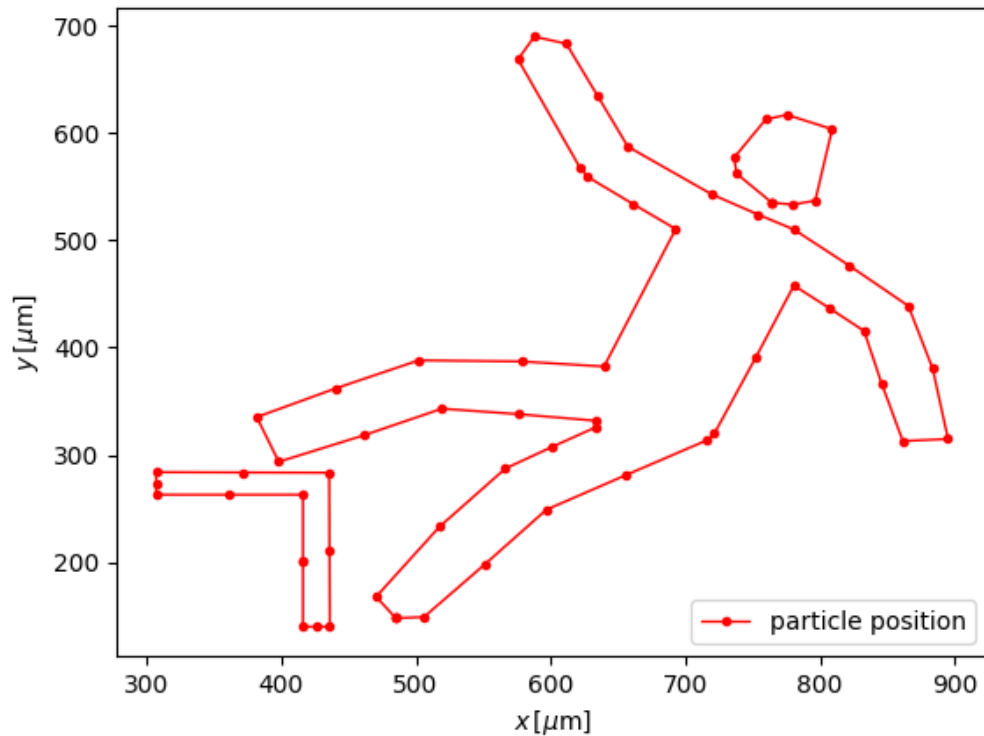
# plt.title('Deterministisches Leiden eines Studierenden-Partikels')
plt.xlabel(r'$x\,, [\mu m]$')
plt.ylabel(r'$y\,, [\mu m]$')
plt.legend(loc='lower right')

plt.show()
plt.savefig(Ausgabe_path/"Meme.png",format="png")

# pattern = r'\d*\.\d'
# coordinates=re.findall(pattern, data)
# x=[]
# y=[]
# for i in range(0, len(coordinates), 2):
#     x.append(coordinates[i])
#     y.append(coordinates[i+1])

# for line in data.strip().splitlines():
#     matches = re.findall(pattern, line)
#     if len(matches) == 2:
#         xvalue = map(float, matches[0])
#         yvalue = map(float, matches[1])
#         x.append(xvalue)
#         y.append(yvalue)

```



```
[153]: log = (
    "[2026-04-21 10:15:30] INFO user=ana at ip=10.0.0.3 bytes=4096\n"
    "[2026-04-21 10:15:31] ERROR user=ben at ip=10.0.0.4 bytes=0\n"
    "[2026-04-21 10:15:32] WARN user=cai at ip=10.0.0.5 bytes=256\n"
)
```

```
# Search / match
```

```
m = re.search(r"user=(\w+)", log)
```

```
print("first user:", m.group(1))
```

```
first user: ana
```

```
[113]: dx=np.diff(x)
dy=np.diff(y)
r_squared=dx**2+dy**2

r_squared_mean=np.mean(r_squared)
print("r_squared_mean=",r_squared_mean)
r_squared_mean_std=np.std(r_squared)/np.sqrt(len(r_squared))
print("r_squared_mean_std=",r_squared_mean_std)
r_squared_mean,r_squared_mean_std,_,_,latexrr=round_to_sigs(r_squared_mean,r_squared_mean_std,r"
```

```
print(latexrr)
```

```
r_squared_mean= 1.3042893390040273  
r_squared_mean_std= 0.11051589949818622  
\qty{1.30(0.12)}{\mu\m\squared}
```

```
[114]: plt.close("all")  
all_data=np.append(dx,dy)  
  
plt.ylabel(r'Wahrscheinlichkeitsdichte')  
plt.xlabel(r'Verschiebung $dx,dy\,,[\mu\,m]$\')  
plt.title(r'Histogramm der Verschiebung $dx,dy$\')
```

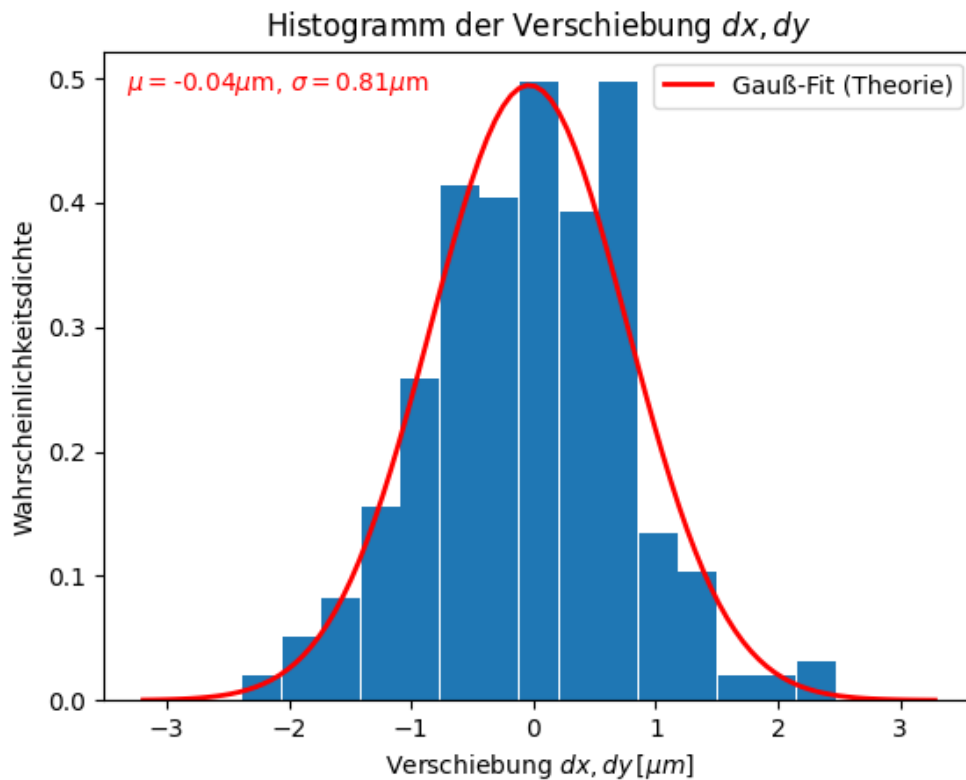
plt.hist(all_data, bins='auto', density=True,rwidth=0.97)

mu=np.mean(all_data)
sigma=np.std(all_data)
x_smooth = np.linspace(min(all_data) - sigma, max(all_data) + sigma, 100)
y_gauss = norm.pdf(x_smooth, mu, sigma)
plt.plot(x_smooth, y_gauss, color='red', linewidth=2, label='Gauß-Fit
↳(Theorie)')

plt.text(0.2, 0.95, rf'\$\mu=\${mu:.2f}\$\mu\$m, \$\sigma=\${sigma:.2f}\$\mu\$m',
↳horizontalalignment='center',verticalalignment='center', transform=plt.gca().
↳transAxes,color='red')

plt.legend()
gauss = norm.pdf(np.linspace(-4,4), mu , sigma)
plt.plot(np.linspace(-4,4), gauss, 'b-', linewidth=2)

plt.show()
plt.savefig(Ausgabe_path/"Nr.2.Histogramm.png",format="png")



```
[115]: a=755/2*1e-9
Delta_a=30/2*1e-9
eta=9.4*1e-4
Delta_eta=0.1*1e-4
r=r_squared_mean*1e-12
Delta_r=r_squared_mean_std*1e-12
T=22.7+273.15
Delta_T=0.3

k_B=3*np.pi*a*eta*r/(2*T)
Delta_k_B=k_B*np.sqrt(Delta_T**2/T**2 + Delta_a**2/a**2 + Delta_eta**2/eta**2 +
    ↪Delta_r**2/r**2)
k_B,Delta_k_B,_,latexkb=round_to_sigs(k_B,Delta_k_B,r'\joule\per\kelvin')
print(k_B)
print(Delta_k_B)

print(latexkb)
```

7.3e-24
8e-25
 $\backslash\text{qty}\{7.3\text{E-}24(8\text{E-}25)\}\{\backslash\text{joule}\backslash\text{per}\backslash\text{kelvin}\}$

```
[116]: D=r_squared_mean/4
Delta_D=r_squared_mean_std/4
print(D,Delta_D)
print(round_to_sigs(D,Delta_D,r'\mikro\m\squared\per\s')[3])
```

```
0.325 0.03
\qty{0.33(0.03)}{\mikro\m\squared\per\s}
```

```
[117]: r_cum=np.cumsum(r_squared)
def lin(x,a,b):
    return a*x + b
popt, pcov = curve_fit(lin,t[:-1],r_cum,absolute_sigma=True)
m,Delta_m,_,latexm=round_to_sigs(popt[0],np.
    ↳sqrt(pcov[0][0]),r'\mikro\m\squared\per\s')
D2,Delta_D2,_,latexD2=round_to_sigs(popt[0]/4,np.sqrt(pcov[0][0])/
    ↳4,r'\mikro\m\squared\per\s')
print(latexm)
print(latexD2)

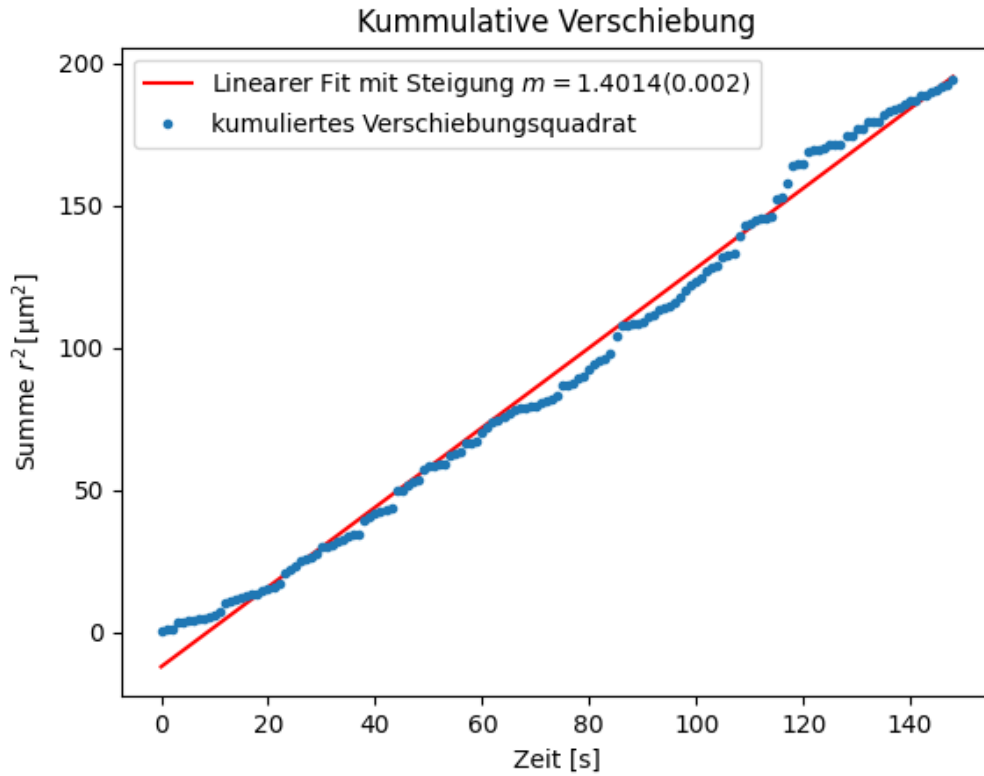
k_B2=3*pi*a*eta*m/(2*T)
Delta_k_B2=k_B*np.sqrt(Delta_T**2/T**2 + Delta_a**2/a**2 + Delta_eta**2/eta**2
    ↳+ Delta_m**2/m**2)
k_B,Delta_k_B2,_,latexkb2=round_to_sigs(k_B,Delta_k_B2,r'\J\K')
print(latexkb2)

print(sigma_abweichung(1.3806485,0,2*0.73,0.08)[1])
print(sigma_abweichung(0.73,0.03,0.73,0.08)[1])
print(sigma_abweichung(0.33,0.03,0.3504,0.0005)[1])

plt.close('all')
plt.plot(t[:-1], lin(t[:-1],*popt), color='red',label=rf'Linearer Fit mit
    ↳Steigung $m={m}({\Delta_m})$')
plt.plot(t[:-1],r_cum, marker='.',linewidth=0,label='kumuliertes
    ↳Verschiebungsquadrat')
plt.xlabel('Zeit [s]')
plt.ylabel(r'Summe $r^2\backslash,[\mathrm{\mu m}^2]$')
plt.title('Kummulative Verschiebung')
plt.legend()

plt.show()
plt.savefig(Ausgabe_path/"Nr.3.kumuliert.png",format="png")
```

```
\qty{1.4014(0.0020)}{\mikro\m\squared\per\s}
\qty{0.3504(0.0005)}{\mikro\m\squared\per\s}
\qty{7.3E-24(4E-25)}{\J\K}
\num{1.0}
\num{0}
\num{0.7}
```



[118]: `gff("6*pi*eta*a*m/(4*T)","eta,a,m,T")`

Funktion:

$$\frac{3\pi a \eta}{2T} m$$

$$\frac{3 \pi a \eta m}{2 T}$$

Absoluter Fehler:

$$\frac{3\pi}{2} \sqrt{\frac{1}{T^4} (\Delta_T^2 a^2 \eta^2 m^2 + T^2 (\Delta_a^2 \eta^2 m^2 + \Delta_\eta^2 a^2 m^2 + \Delta_m^2 a^2 \eta^2))}$$

$$\frac{3 \pi \sqrt{\frac{\Delta_T^2}{T^4} a^2 \eta^2 m^2 + T^2 (\Delta_a^2 \eta^2 m^2 + \Delta_\eta^2 a^2 m^2 + \Delta_m^2 a^2 \eta^2)}}{2}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_T^2}{T^2} + \frac{\Delta_a^2}{a^2} + \frac{\Delta_\eta^2}{\eta^2} + \frac{\Delta_m^2}{m^2}}$$

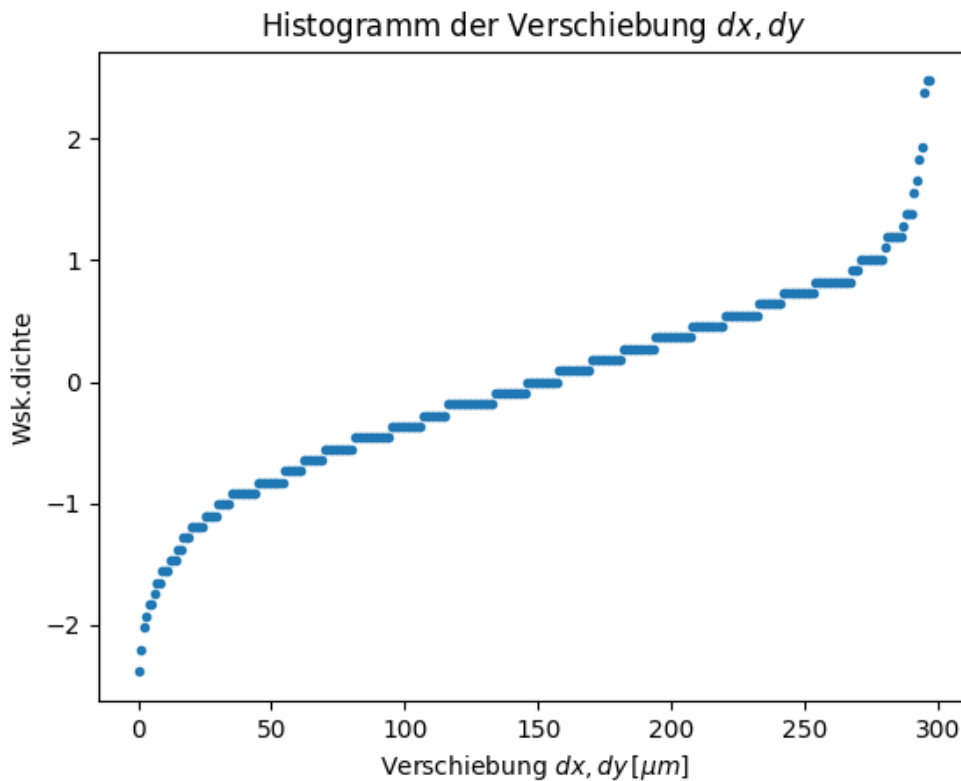
$$\sqrt{\frac{\Delta_T^2}{T^2} + \frac{\Delta_a^2}{a^2} + \frac{\Delta_\eta^2}{\eta^2} + \frac{\Delta_m^2}{m^2}}$$


```
[118]: (3*pi*a*eta*m/(2*T),
        3*pi*sqrt((Delta_T**2*a**2*eta**2*m**2 + T**2*(Delta_a**2*eta**2*m**2 +
Delta_eta**2*a**2*m**2 + Delta_m**2*a**2*eta**2))/T**4)/2,
        sqrt(Delta_T**2/T**2 + Delta_a**2/a**2 + Delta_eta**2/eta**2 +
Delta_m**2/m**2),
        [(eta, Delta_eta), (a, Delta_a), (m, Delta_m), (T, Delta_T)])
```

```
[119]: plt.close("all")
all_data=np.append(dx,dy)
plt.ylabel(r'Wsk.dichte')
plt.xlabel(r'Verschiebung $dx,dy\,[\mu m]$')
plt.title(r'Histogramm der Verschiebung $dx,dy$')

plt.plot(np.sort(all_data),marker='.',linestyle='')

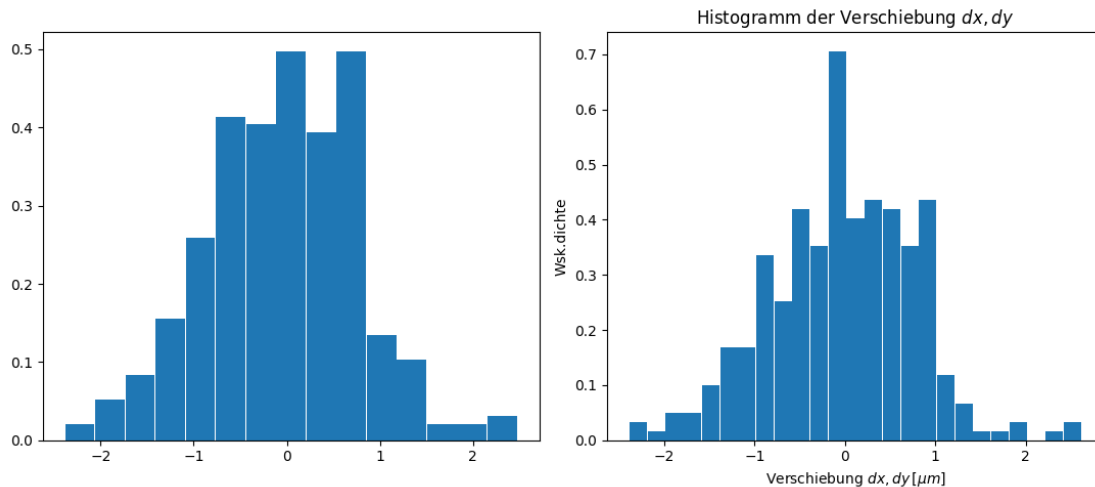
mu=np.mean(all_data)
sigma=np.std(all_data)
# gauss = norm.pdf(np.linspace(-4,4), mu , sigma)
# plt.plot(np.linspace(-4,4), gauss, 'b-', linewidth=2)
plt.show()
```



```
[120]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11,5))

plt.ylabel(r'Wsk.dichte')
plt.xlabel(r'Verschiebung $dx,dy\,([\mu m])$')
plt.title(r'Histogramm der Verschiebung $dx,dy$')
binwidth=0.2
ax1.hist(all_data, bins="auto", density=True, rwidth=0.97)
ax2.hist(all_data, bins=np.arange(min(all_data), max(all_data) + binwidth,
binwidth), density=True, rwidth=0.97)

mu=np.mean(all_data)
sigma=np.std(all_data)
# gauss = norm.pdf(np.linspace(-4,4), mu , sigma)
# plt.plot(np.linspace(-4,4), gauss, 'b-', linewidth=2)
plt.tight_layout()
plt.show()
```



```
[121]: compare_table_withliterature(r'k_B',r'k_B',r'\J\per\K',0.73,0.08,1.3806485)
```

```
\begin{table}[htb]
\centering
\caption{}
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
\toprule
k_B[\unit{\J\per\K}]&k_B[\unit{\J\per\K}]&\text{abs.Abw.}[\unit{\J\per\K}]
&\text{proz.Abw.}[\unit{\percent}]&\&S[\sigma]\midrule
\text{num}{0.73(0.08)}&\text{num}{1.3806485}&\text{num}{0.65(0.08)}&\text{num}{47(8)}&\text{num}{9}
\bottomrule
\end{tabularx}
\end{table}
```

```
\label{table:Vergleich_k_B}
\end{table}
```